

# **Shortest Path Algorithms on Turkish Cities Network**

COMP201 – Term Project

Group 5 Members: Beren Keçe, Anıl Koca, Deniz Durgan, Aylin Öncü, Selin Öker

## **Table of Contents**

1. Introduction
2. Graph Representation and Data Structures
  - 2.1 Custom List Implementation
  - 2.2 Stack Interface Implementation
  - 2.3 Graph Class Implementation
  - 2.4 Dataset Loading and Graph Construction
3. Depth-First Search (DFS)
4. Modified Depth-First Search (DFS-Shortest)
5. Dijkstra's Algorithm
6. Graphical User Interface (GUI)
7. Comparative Study
8. Time Complexity Analysis
9. Empirical Results
10. Discussion
11. References
12. Group Member Contributions

## 1. Introduction

In this project, we study the shortest path problem in weighted graphs using real-world distance data between Turkish cities. The objective is to analyze and compare different algorithms in terms of their ability to find optimal paths and their performance characteristics.

Three algorithms are implemented and evaluated: Depth-First Search (DFS), a modified version of DFS designed to find the shortest path, and Dijkstra's algorithm. The comparison is based on path correctness, total distance, and empirical execution time.

## 2. Graph Representation and Data Structures

### 2.1 Custom List Implementation

Our Array-List Implementation:

We needed a custom List implementation for implementing an adjacency list in our Graph class which supports both Depth-First Search and Modified Depth-First Search algorithms. An array list structure was chosen because it provides several advantages such as automatic dynamic resizing, constant-time indexing, and efficient appending of new elements. These characteristics make the structure well suited for representing adjacency lists, where fast access to neighbors and quick list expansion are critical for search performance.

The class has two data fields:

```
private E[] data;  
private int size = 0;
```

**Figure 1.** Data fields of the custom ArrayList implementation

The array “data” stores the elements, while the integer variable “size” keeps track of the number of elements currently stored in the array.

The class has two constructors:

```
public ArrayList(int initialCapacity) {  
    if (initialCapacity <= 0)  
        initialCapacity = 10;  
    data = (E[]) new Object[initialCapacity];  
}  
  
public ArrayList() {  
    this(initialCapacity: 10); // default capacity of 10  
}
```

**Figure 2.** Code snippet showing the two constructors of the custom ArrayList implementation.

The first constructor takes an integer parameter representing the initial capacity of the array and initializes the array accordingly. The second constructor does not take any parameters and internally calls the first constructor with a default initial capacity of 10.

The class includes thirteen methods: `add()`, `set()`, `get()`, `removeLast()`, `remove()`, `isEmpty()`, `contains()`, `increaseCapacity()`, `size()`, `capacity()`, `reverse()`, `checkIndex(int index)`, and `toString()`.

The `increaseCapacity` method is a private helper function that is called within the `add()` method when the array is full. When invoked, it creates a new array with double the previous capacity and copies all existing elements into the new array. Since array variables in Java act as references, the new array replaces the old one, allowing the garbage collector to clean up the unused memory.

```
//Increases the size of the array if it is full
private void IncreaseCapacity() {
    if (size == data.length) {
        int newCapacity = data.length * 2;
        E[] newArray = (E[]) new Object[newCapacity];
        // copy old values
        for (int i = 0; i < data.length; i++) {
            newArray[i] = data[i];
        }
        data = newArray;
    }
}
```

**Figure 3.** Code snippet demonstrating the dynamic resizing mechanism used in the custom ArrayList implementation.

## 2.2 Stack Interface Implementation

The Stack interface defines the fundamental operations of a stack data structure using a generic type parameter *E*. A stack follows the Last-In, First-Out (LIFO) principle, meaning that the most recently added element is the first one to be removed. This interface provides a clear and consistent contract for stack implementations used throughout the project.

By defining the stack as an interface rather than a concrete class, the design promotes abstraction and modularity. Different stack implementations can be developed and substituted without affecting the algorithms that rely on stack operations. This approach improves code readability, maintainability, and flexibility.

The Stack interface supports the following core operations:

- **pushCity(E element):** Adds a new element to the top of the stack, making it the next element to be removed.
- **popCity():** Removes and returns the element at the top of the stack, following the LIFO principle.
- **peekCity():** Returns the top element without removing it, allowing inspection of the next element to be popped.
- **isEmpty():** Checks whether the stack contains any elements.
- **size():** Returns the number of elements currently stored in the stack.

This stack interface is primarily used in the DFS-based algorithms to manage the traversal order of cities during path exploration.

## 2.3 Graph Class Implementation

The Graph class represents a weighted graph structure used to model the relationships between cities and the distances between them. The graph is implemented using an adjacency list representation, which is efficient for sparse graphs and allows fast access to neighboring cities.

The core data structures of the graph consist of three main fields. The cities ArrayList stores the names of all cities in the graph, allowing each city to be referenced by its index. The adj ArrayList represents the adjacency list, where each index corresponds to a city in the cities list, and each element is another ArrayList containing edge information for neighboring cities. The cityCount variable keeps track of the total number of cities currently stored in the graph and ensures consistent indexing.

The constructor of the Graph class initializes these data structures by creating empty lists for cities and adjacency information and setting the initial city count to zero. This establishes the base state of the graph before any cities or connections are added.

The Graph class provides several methods to manage cities and connections:

- **ensureCity(String name):** Checks whether a city exists in the graph; if not, it adds the city and initializes its adjacency list, returning the city's index.
- **getCityIndex(String name):** Returns the index of a city by name, or  $-1$  if the city does not exist.
- **addEdge(String from, String to, int distance):** Adds a directed edge between two cities with the specified distance.
- **addUndirectedEdge(String a, String b, int distance):** Creates a bidirectional connection by inserting edges in both directions.
- **getNeighbors(String city):** Returns the list of neighboring cities; returns an empty list if the city is not found.
- **getDistance(String from, String to):** Returns the distance between two cities if a direct edge exists; otherwise returns **99999** to indicate no direct connection.
- **getCities():** Returns the list of all city names stored in the graph.

To represent connections between cities, the Graph class includes a static inner Edge class. Each Edge object stores the destination city name and the distance associated with that connection. Declaring this class as static allows edge objects to function independently of any specific Graph instance and serve purely as data holders within the adjacency list.

## 2.4 Dataset Loading and Graph Construction

CityCSVLoader Class Implementation:

The purpose of this class is to provide necessary functions to read the "Turkish cities.csv" dataset file and construct the adjacency list using the Graph class based on the cities and their distances.

We import `import java.io.BufferedReader;`, `import java.io.FileReader;` since they provide the necessary functions to read from a .csv file.

After creating an instance of buffered reader, i created a variable called header and assign the first line into it. After that as shown in the image below, i created a string array `headerTokens` which stores the tokenized version of the header by the delimiter comma. After that we construct a new array with the size=`headerTokens.length-1`. The reason that we decreased it by one is because in the dataset file, first cell (when separated by the delimiter comma) is empty. And with the help of a for loop we assign each value (except the first one since it is empty) of the `headerTokens` into that new array.

```
if (header == null) {  
    return g;  
}  
//When separating each word/number in each line, we use a comma as the delimiter  
String[] headerTokens = header.split(regex: ",");  
int n = headerTokens.length - 1; //First cell is empty  
String[] cols = new String[n];  
  
for (int i = 1; i < headerTokens.length; i++) {  
    cols[i-1] = headerTokens[i].trim();  
    g.addEdge(cols[i-1], cols[i-1], distance: 0);  
}
```

**Figure 4.** Parsing and processing of the CSV header to extract city names for graph construction.

After reading the header and extracting each city name, ensuring that every city is registered in the graph by inserting a self edge of distance zero. It then processes each subsequent row by reading the row city name and iterating through all distance values. For each valid distance, it creates an undirected edge between the row city and the corresponding column city. If a value cannot be parsed or equals to 99999, the loader treats it as an unreachable path. This design ensures that all cities and connections are accurately represented in the adjacency list. Additionally, by automatically registering cities and safely handling malformed input, the loader guarantees that the resulting graph is fully compatible with the DFS and modifies DFS algorithms and accurately reflects the connectivity defined in the dataset file.

### 3. Depth-First Search (DFS)

Depth-First Search Implementation:

Depth-First Search (DFS) is a fundamental algorithm used to traverse or search through graph structures. In this project, DFS is implemented to find a path between two cities in a network represented as a graph. The algorithm explores as far as possible along each branch before backtracking, making it suitable for pathfinding in scenarios where memory usage is constrained or when we want to explore all possible paths in depth-first order. In this version of our implementation, the algorithm stops when it finds a path and that path does not have to be the shortest.

In this implementation, a stack is used to manage nodes to explore, and two lists are maintained: one to track visited cities and another to record parent cities for path reconstruction.

```

ArrayStack<String> stack = new ArrayStack<>(graph.getCities().size()); //The next city to explore is always the one on top of the stack
ArrayList<String> visited = new ArrayList<>(); //all cities that have already been visited
ArrayList<String> parentCity = new ArrayList<>(); //parent city for each visited city

```

**Figure 5.** Initialization of the stack and auxiliary lists used in the DFS algorithm.

The stack stores the cities that will be explore, and the element on top of the stack is always the next city to be explored. ArrayList visited stores the cities that have been visited. It is use to avoid visiting the same city twice.

ArrayList parentCity stores the parent cities of each visited city. When you reach the target city, you can reconstruct the path by following parent links from the target back to the start. The algorithm outputs the path found from the start city to the target city, along with the total distance traveled.

Function dfs handles the traversal part:

```

public void dfs(String start, String target){

    ArrayStack<String> stack = new ArrayStack<>(graph.getCities().size()); //The next city to explore is always the one on top of the stack
    ArrayList<String> visited = new ArrayList<>(); //all cities that have already been visited
    ArrayList<String> parentCity = new ArrayList<>(); //parent city for each visited city

    //Push start
    stack.pushCity(start);
    visited.add(start);
    parentCity.add(element: null);

    while(!stack.isEmpty()){

        String current = stack.popCity();

        if(current.equals(target)){
            printPath(visited, parentCity, start, target);
            return;
        }

        //Expand neighbors
        ArrayList<Graph.Edge> neighbors = graph.getNeighbors(current);

        for(int i = 0; i < neighbors.size(); i++){
            Graph.Edge e = neighbors.get(i);

            if(!visited.contains(e.to)){
                visited.add(e.to);
                parentCity.add(current);
                stack.pushCity(e.to);
            }
        }

        System.out.println( x: "No path found.");
    }
}

```

**Figure 6.** Implementation of the DFS traversal process using a stack and visited tracking.

The traversal process in the DFSPathFinder algorithm begins by initializing a stack with the start city and marking it as visited. The stack ensures that cities are explored in a last-in, first-out (LIFO) order, which is the fundamental principle of depth-first search. In each iteration of the traversal loop, the algorithm pops the top city from the stack and checks whether it matches the target city. If the target is not yet reached, the algorithm retrieves all neighbor cities of the current city. Each neighbor is examined to see if it has already been visited; if not, it is added to the visited list, its parent city is recorded for path reconstruction, and it is pushed onto the stack for future exploration. This process continues until the stack is empty or the target city is found, effectively exploring as deeply as possible along each path before backtracking to explore alternative routes.

Function dfs handles the traversal part:

```

private void printPath(ArrayList<String> visited, ArrayList<String> parent,
    String start, String target){

    ArrayList<String> path = new ArrayList<>();
    int totalDistance = 0;

    String current = target;

    while (current != null) {
        path.add(current);
        String previous = getParentOf(current, visited, parent);

        if(previous != null){
            totalDistance += graph.getDistance(previous, current);
        }

        current = previous;
    }

    path.reverse();
    System.out.println("DFS PATH: " + path.toString());
    System.out.println("TOTAL DISTANCE: " + totalDistance + "km");
}

private String getParentOf(String city, ArrayList<String> visited, ArrayList<String> parent){
    for(int i = 0; i < visited.size(); i++){
        if(visited.get(i).equals(city)){
            return parent.get(i);
        }
    }
    return null;
}

```

**Figure 7.** Reconstruction of the DFS path and computation of the total traveled distance.

Once the target city is located, the algorithm reconstructs the path from the start city to the target using the parent city information stored during traversal. Starting from the target city, the algorithm traces back through each city's parent until it reaches the start city, building the path in reverse order. During this process, the total distance traveled is calculated by summing the distances between consecutive cities using the graph's distance information.

After reaching the start city, the reconstructed path is reversed to display the correct order from start to target. Finally, both the path and the total distance are presented as output, providing a clear representation of the route discovered by the DFS traversal.

The DFSPathFinder implementation provides a practical approach to exploring city networks using depth-first search. By maintaining visited and parent lists, it reconstructs the path and calculates the total distance effectively.

#### 4. Modified Depth-First Search (DFS-Shortest)

Modified Depth-First Search Implementation:

This algorithm extends the standard Depth-First Search approach to find the shortest path between two cities in a weighted graph. Unlike standard DFS, which stops after finding the first path, this modified version explores all possible paths while keeping track of the shortest one.

A private State class is used to represent the current search state. Each state contains the current city, the path taken so far, and the total distance traveled. The path list prevents cycles by ensuring that a city is not revisited within the same path. The distance value allows the algorithm to compare paths and prune those that are already longer than the best path found so far.

The traversal begins by pushing an initial state containing the start city, an initial path, and a distance of zero onto the stack. During each iteration, the algorithm pops the top state and checks whether the current city is the target city. If so, the distance of this path is compared with the best distance found so far, and the best path is updated if necessary.

If the target city is not reached, the algorithm examines all neighboring cities. For each neighbor that is not already present in the current path, a new state is created using a deep copy of the current path. The neighbor is added to the path, and the distance is updated accordingly. This new state is pushed onto the stack only if its distance is smaller than the current best distance, effectively pruning inefficient paths.

Path reconstruction is integrated directly into the traversal process, eliminating the need for backtracking using parent pointers. After the traversal completes, the algorithm outputs the shortest path and its total distance.

## 5. Dijkstra's Algorithm

Purpose of the Algorithm:

Dijkstra's algorithm is a widely used shortest path algorithm designed for weighted graphs. In this project, it is used to find the shortest path between two cities in the Turkish cities dataset.

The algorithm works by repeatedly selecting the closest unvisited city and updating the distance values of its neighboring cities. Unlike DFS, which may return a suboptimal path, Dijkstra's algorithm guarantees the minimum distance path.

During execution, distance values are updated dynamically, and parent information is stored for each city. After the algorithm finishes, this parent information is used to reconstruct the shortest path from the start city to the target city.

```
// this helps us keep track of a city and how far it is
private static class CityDistance implements Comparable<CityDistance> {
    int cityIndex;
    double distance;
    public CityDistance(int cityIndex, double distance) {
        this.cityIndex = cityIndex;
        this.distance = distance;
    }
    // compare cities by distance (smaller is better)
    @Override
    public int compareTo(CityDistance other) {
        return Double.compare(this.distance, other.distance);
    }
}
```

**Figure 8.** CityDistance class implementing Comparable to allow distance-based ordering of cities in the priority queue.



In this implementation, a data structure called `CityDistance` is used to store a city together with its current distance from the start city. Storing this information as a single object simplifies distance tracking during the algorithm's execution.

The `CityDistance` class implements the `Comparable` interface, allowing cities to be compared based on their distance values. This enables the algorithm to always select the city with the smallest distance, which is a core requirement of Dijkstra's algorithm.

The graph is constructed by reading distance data from a CSV file. These distances are stored in an adjacency matrix, where very large values represent the absence of a direct road between two cities.

```
// DIJKSTRA'S ALGORITHM: finds shortest paths from one city to all others
public DijkstraResult findShortestPaths(int sourceIndex) {
    long start = System.nanoTime(); // start timer
    double[] distance = new double[numCities]; // how far to each city
    Arrays.fill(distance, Double.POSITIVE_INFINITY); // start with infinity
    distance[sourceIndex] = 0.0; // starting city is 0 km away
    int[] predecessor = new int[numCities]; // tracks the path
    Arrays.fill(predecessor, -1); // -1 means no path yet
    boolean[] visited = new boolean[numCities]; // marks cities we checked
```

**Figure 9.** Initialization of distance, predecessor, and visited arrays in the Dijkstra algorithm.

At the beginning of the algorithm, a distance array is created to store the shortest distance from the starting city to every other city. All distance values are initially set to infinity, since no paths are known at the start. The distance of the source city is then set to zero because the algorithm begins from this city.

A predecessor array is initialized to keep track of the previous city on the shortest path. All values are initially set to `-1`, indicating that no path has been discovered yet.

In addition, a visited array is used to mark cities that have already been processed. This prevents the algorithm from revisiting the same city multiple times and improves efficiency.

```
PriorityQueue<CityDistance> pq = new PriorityQueue<>(); // priority queue (min heap)
pq.offer(new CityDistance(sourceIndex, 0.0)); // add starting city
```

**Figure 10.** Priority queue initialization and insertion of the source city.

A priority queue is used to ensure that the city with the smallest distance value is always selected next. The priority queue functions as a min-heap, meaning that cities with lower distance values are given higher priority during processing.

The starting city is initially added to the priority queue with a distance value of zero, reflecting that the algorithm begins from this city..

```
while (!pq.isEmpty()) { // keep going until done
    CityDistance current = pq.poll(); // get closest city
    int u = current.cityIndex;
    if (visited[u]) continue; // skip if already checked
    visited[u] = true; // mark as checked
```

**Figure 11.** Selection and visitation of the closest unvisited city using the priority queue.

The algorithm continues while the priority queue is not empty. In each iteration, the city with the smallest distance value is removed from the priority queue using the poll() operation.

After a city is selected, its index is checked. If the city has already been visited, it is skipped to avoid redundant processing. Otherwise, the city is marked as visited, indicating that the shortest distance to this city has been finalized.

Once marked as visited, the algorithm proceeds by examining the neighboring cities of the current city and updating their distance values when necessary.

```
for (int v = 0; v < numCities; v++) { // check all neighbors
    if (u == v || adjacencyMatrix[u][v] >= 99999 || visited[v]) continue; // skip if no road
    double newDist = distance[u] + adjacencyMatrix[u][v]; // calculate new distance
    if (newDist < distance[v]) { // found shorter path?
        distance[v] = newDist; // update distance
        predecessor[v] = u; // remember path
        pq.offer(new CityDistance(v, newDist)); // add to queue
    }
}
```

**Figure 12.** Updating distance and predecessor information for neighboring cities.

In this stage of the algorithm, all neighboring cities of the current city are examined using the adjacency matrix. The matrix stores the distances between cities, while very large values (such as 99999) indicate the absence of a direct road. Cities that are unreachable or already visited are skipped.

For each valid neighboring city, a new distance is calculated by adding the distance of the current city to the edge weight obtained from the adjacency matrix. If this newly computed distance is smaller than the previously stored distance, a shorter path has been found.

When a shorter path is detected, both the distance array and the predecessor array are updated accordingly. The neighboring city is then added back to the priority queue using the offer() operation so that it can be processed again with its updated distance.

This process continues until the priority queue becomes empty, indicating that all reachable cities have been processed. At this point, the shortest distances from the source city to all other cities are finalized. Since this implementation computes shortest paths from a single source to all cities, no early stopping based on a specific target city is applied.

After the algorithm completes, the shortest path is reconstructed using the predecessor information stored during execution. Path reconstruction is not performed directly inside the Dijkstra algorithm itself.

The predecessor array is used to trace the path backward from the destination city to the source city. This path is then reversed to obtain the correct order from the start city to the target city. If a city is unreachable from the source, an empty path is returned.

## 6. Graphical User Interface (GUI)

### GUI Implementation:

The GUI class provides a graphical user interface that allows the user to enter a start city and a destination city, run multiple path-finding algorithms, and display their results in a single window.

```
private static Graph graph;  
private static Dijkstra dijkstra;  
private static DFSPathFinder dfs;  
private static DFSShortestPathFinder dfsShortest;
```

**Figure 13.** Static data fields used in the GUI class

These static data fields store the core objects used by the GUI to perform path-finding operations. The graph variable represents the city network constructed from the dataset and is shared across the entire application. The dijkstra, dfs, and dfsShortest objects hold instances of the three different path-finding algorithms (Dijkstra, standard DFS, and modified DFS for shortest path). Declaring them as static allows them to be accessed directly from the main method and other static methods in the GUI class without creating a GUI object, ensuring that all components of the interface use the same graph and algorithm instances consistently

```
//load and initialize algorithms for finding path  
try {  
    graph = CityCSVLoader.loadFromDistanceMatrix("Turkish cities.csv");  
    dijkstra = new Dijkstra("Turkish cities.csv");  
    dfs = new DFSPathFinder(graph);  
    dfsShortest = new DFSShortestPathFinder(graph);  
}  
  
//exception if dataset loading fails  
catch (Exception e) {  
    JOptionPane.showMessageDialog(null, "Error while loading the dataset: "+e.getMessage());  
    return;  
}
```

**Figure 14.** Dataset loading and initialization of path-finding algorithms

This try-catch block is responsible for loading the dataset and initializing all pathfinding components used by the GUI. Inside the try block, the graph is first constructed by reading the distance matrix from the Turkish cities.csv file using CityCSVLoader, after which the Dijkstra, DFS, and modified DFS (shortest path) algorithm objects are created and prepared for use. If any error occurs during file reading, parsing, or object initialization, the catch block is triggered. In that case, an error message is displayed to the user via a JOptionPane, and the program terminates early using return to prevent the GUI from running with incomplete or invalid data.

```

String startCityInput = startField.getText().trim();
String destinationCityInput = destinationField.getText().trim();

//check if input fields are empty
if (startCityInput.isEmpty() || destinationCityInput.isEmpty()) {
    resultArea.setText("Please enter both cities.");
    return;
}

//case insensitive handling of user input
String startCity = makeCityNameCaseInsensitive(startCityInput);
String destCity = makeCityNameCaseInsensitive(destinationCityInput);

if (startCity == null || destCity == null) {
    resultArea.setText("One or both cities not found in the database.");
    return;
}

```

**Figure 15.** User input validation and case-insensitive city matching

This code segment reads the start and destination city names entered by the user, removes any leading or trailing spaces, and first checks whether either input field is empty, displaying an error message if so. It then converts the user input into a case-insensitive match against the cities stored in the graph by calling `makeCityNameCaseInsensitive`, ensuring that different letter cases do not cause lookup errors. If either city cannot be found in the dataset, the program informs the user that one or both city names are invalid and stops further execution to prevent running the algorithms with incorrect input.

```

// Redirect System.out to capture algorithm output
PrintStream originalOut = System.out;
ByteArrayOutputStream baos = new ByteArrayOutputStream();
System.setOut(new PrintStream(baos));

```

**Figure 16.** Redirecting console output to the GUI

This code temporarily redirects the standard console output (`System.out`) to an in-memory buffer so that the output produced by the algorithms can be captured and displayed inside the GUI instead of the terminal. First, the original `System.out` reference is saved in `originalOut` so it can be restored later. Then, a `ByteArrayOutputStream` is created to store all printed output in memory. By setting `System.out` to a new `PrintStream` that writes to this buffer, any `System.out.println()` calls executed afterward are collected in `baos`, allowing the program to retrieve the output as a string and show it in the GUI's result area.

```

try {
    //run path finding algorithms and display results
    System.out.println("DFS Algorithm: ");
    dfs.dfs(startCity, destCity);

    System.out.println("\nDFS (Shortest) Algorithm: ");
    dfsShortest.dfsShortest(startCity, destCity);

    System.out.println("\nDijkstra's Algorithm: ");
    Dijkstra.DijkstraResult dResult = dijkstra.findShortestPaths(startCity);
    int destIdx = dijkstra.getCityIndex(destCity);

    System.out.println("Path: " + dResult.getPathString(destIdx));
    System.out.println("Distance: " + (int)dResult.getDistance(destIdx) + " km");

    resultArea.setText(baos.toString());
    resultArea.setCaretPosition(0);

} catch (Exception ex) {
    resultArea.setText("Error during calculation: " + ex.getMessage());
} finally {
    System.setOut(originalOut); // Reset console output
}

```

**Figure 17.** Execution of path-finding algorithms and result display in the GUI

This code block is designed to execute the DFS, DFS (Shortest), and Dijkstra algorithms and display their results within the GUI. Before executing each algorithm, a heading is printed using `System.out.println`, and all output is captured in a `ByteArrayOutputStream` to redirect it from the console to memory. For the Dijkstra algorithm, the shortest path and distance for the target city are obtained through specific methods and similarly recorded in the buffer. The collected output is then displayed in the GUI's result area via `resultArea.setText(baos.toString())`, and `setCaretPosition(0)` ensures that the text view starts from the top. Any exceptions encountered during execution are handled in the catchblock, while the finally block restores `System.out` to its original state, preserving normal console output for the remainder of the program.

## 7. Comparative Study

In this section, the three implemented algorithms—DFS, DFS-Shortest, and Dijkstra's algorithm—are compared based on path correctness, total path distance, and execution time. Several city pairs were selected from the dataset such that the cities were not directly connected, allowing a meaningful evaluation of each algorithm's behavior.

The results indicate that standard DFS is generally fast due to its early termination strategy; however, it does not guarantee finding the shortest path. In many cases, DFS returns a valid path with a longer total distance compared to the optimal solution.

DFS-Shortest consistently produces the shortest path by exploring all possible routes between the source and destination cities. While this guarantees correctness, it significantly increases

execution time, especially as the number of possible paths grows. This makes DFS-Shortest less practical for larger or denser graphs.

Dijkstra's algorithm achieves a balance between correctness and performance. By always selecting the closest unvisited city, it avoids unnecessary exploration while still guaranteeing the shortest path. As a result, Dijkstra's algorithm outperforms DFS-Shortest in terms of execution time while maintaining optimal path results.

Overall, the comparative results show that although each algorithm has strengths, Dijkstra's algorithm is the most suitable choice for shortest path problems in weighted graphs such as the Turkish cities network.

## 8. Time Complexity Analysis

In the given dataset, the graph contains 17 cities ( $V = 17$ ).

Edges (E) represent the valid connections between cities where the distance value is not equal to 99999.

Theoretical time complexity analysis is independent of these exact values, since Big-O notation describes how algorithms scale with input size rather than their performance on a specific dataset.

Depth-First Search (DFS) has a time complexity of  $O(V + E)$  because, in the worst case, it may visit all cities and all connections.

The Modified DFS (DFS-Shortest) algorithm also has a worst-case time complexity of  $O(V + E)$ . Although it avoids some unnecessary paths using pruning, it can still explore a large portion of the graph.

Dijkstra's Algorithm has a time complexity of  $O((V + E) \log V)$  when implemented with a priority queue, due to the cost of selecting the closest unvisited city at each step.

## 9. Empirical Results

|                        | DFS   | DFS shortest | Dijkstra |
|------------------------|-------|--------------|----------|
| Istanbul - Adana       | 5.894 | 8.314        | 4.568    |
| Istanbul - Ankara      | 4.062 | 15.912       | 2.382    |
| Ankara - Izmir         | 4.616 | 51.947       | 3.398    |
| Ankara - Konya         | 4.148 | 7.147        | 2.503    |
| Izmir - Antalya        | 4.047 | 3.095        | 2.062    |
| Adana - Mersin         | 4.377 | 13.210       | 3.436    |
| Gaziantep - Diyarbakir | 3.578 | 2.603        | 2.284    |
| Konya - Antalya        | 3.606 | 22.821       | 2.229    |

The table below shows the measured execution times of DFS, DFS-Shortest, and Dijkstra's Algorithm for eight different city pairs. The city pairs were chosen such that a valid path exists between the selected cities. Each pair was tested 10 times, the table shows the average results.

The table shows that Dijkstra's Algorithm consistently runs faster than the other two methods. This is expected, as Dijkstra's approach directly focuses on the shortest paths instead of exploring many alternatives.

Standard DFS generally runs in a reasonable time, but it does not guarantee the shortest path, which limits its usefulness for this problem.

DFS-Shortest always finds the optimal path, but its runtime varies significantly depending on how many alternative paths are explored. In some cases, this leads to noticeably higher execution times.

Overall, the empirical results are consistent with the theoretical time complexity analysis.

## **10. Discussion**

This project demonstrates how algorithm choice directly affects both the accuracy and efficiency of shortest path solutions. Although DFS is simple to implement and requires relatively low memory, it is not suitable for problems where optimal solutions are required, as it may terminate with suboptimal paths.

The modified DFS approach improves upon standard DFS by guaranteeing the shortest path; however, this improvement comes at the cost of significantly increased execution time. As the number of cities and possible paths increases, this approach becomes less scalable and impractical for real-world applications. Dijkstra's algorithm provides the most balanced solution among the three methods. Its greedy strategy ensures correctness while avoiding unnecessary exploration, making it well suited for weighted graphs with non-negative edge weights. The empirical results confirm that Dijkstra's algorithm scales better and performs more efficiently than DFS-based approaches in the given dataset.

Working with real-world distance data also highlights the importance of scalability in algorithm design. As graph size grows, exhaustive search-based algorithms become less feasible, emphasizing the need for efficient and well-structured algorithms.

Overall, this project shows how theoretical algorithm properties translate directly into practical performance in real applications.

## **11. References**

- [1] COMP201 Course Slides, "Graph Algorithms and Shortest Path Algorithms," MEF University, 2024.
- [2] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, Introduction to Algorithms, 3rd ed., MIT Press, 2009.

## **12. Group Member Contributions**

### **Group Member Contributions**

Beren Keçe was responsible for implementing the custom stack and graph data structures, as well as the user interface.

Anıl Koca implemented the Depth-First Search (DFS) and the modified DFS (DFS-Shortest) algorithms.

Deniz Durgan implemented Dijkstra's algorithm.

Aylin Öncü conducted the theoretical and empirical time complexity analysis.

Selin Öker was responsible for the comparative study, report organization, and presentation preparation.